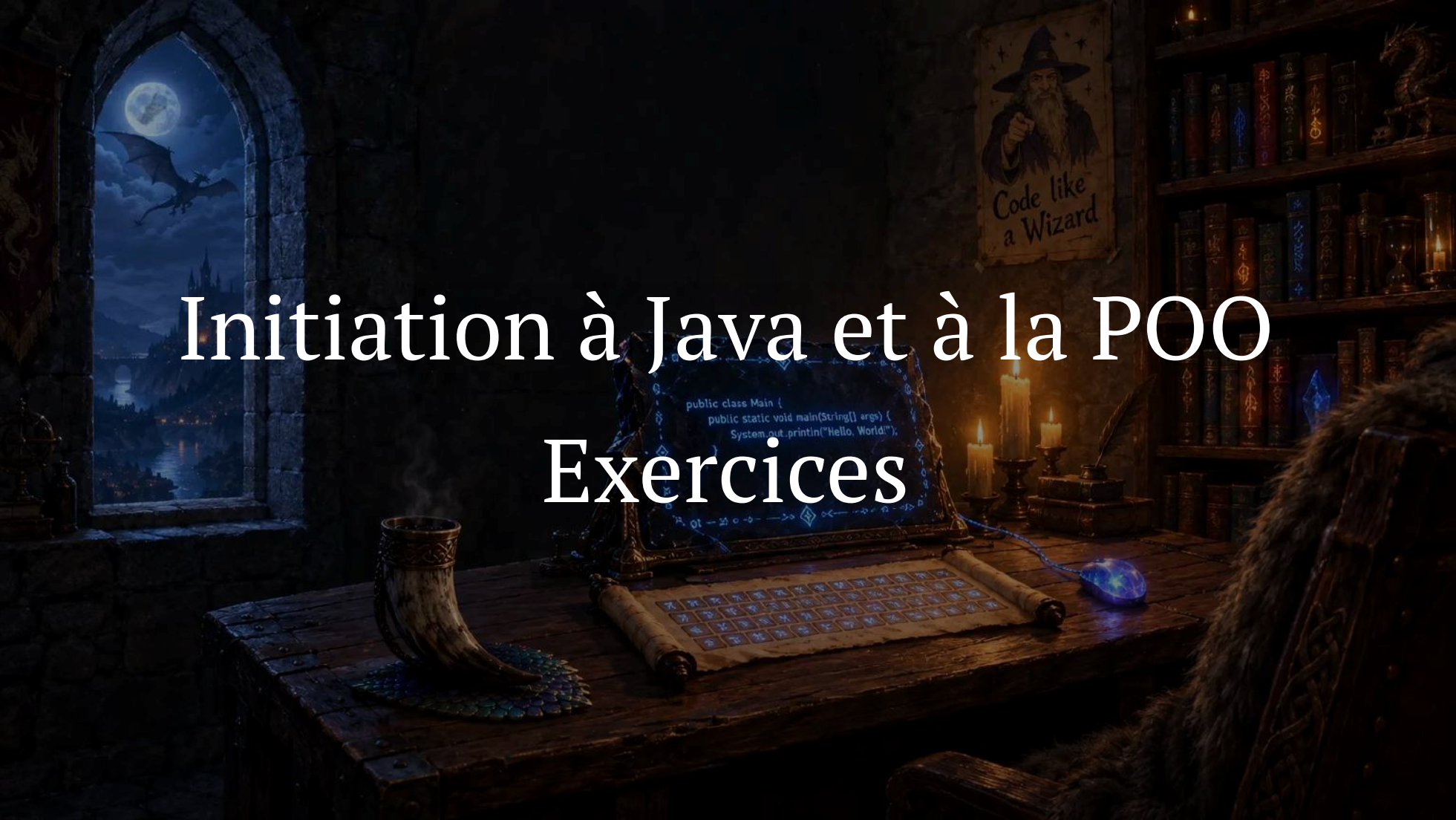



# Initiation à Java et à la POO

## Exercices

# 🎯 Atelier Java – 1h30

## Choisissez votre mini-projet !

Réalisation d'un mini-projet **console**, au choix parmi 3 propositions.

1. 👛 Projet 1 : **Carnet de contacts interactif**
2. 🗒️ Projet 2 : **Système de notes d'étudiants**
3. 🐶 Projet 3 : **Gestion d'un refuge pour animaux**

Chaque projet mobilise :

✅ Les **classes et objets**
✅ Les **collections** ( `List` , `Map` )
✅ La **manipulation de fichiers** ou la **gestion d'erreurs**
✅ Les **conditions, boucles**, et la **console**

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

## 🧠 Votre mission :

- Lire attentivement l'énoncé.
- Organiser votre code en classes claires.
- Tester régulièrement votre programme.
- Bonus : améliorez l'interface console ou ajoutez des fonctionnalités !

📌 **Objectif :** Appliquer vos connaissances dans un projet concret et fonctionnel.

> ℹ️ **Rappel** : Une application en mode console **n'est pas une application CLI** (Command Line Interface).
> Ici, on attend **un menu simple** et des interactions avec l'utilisateur via `Scanner` dans la console, **pas** une app avec des options en ligne de commande comme `java MonApp -f fichier.txt` .

# 🎒 Projet 1 : **Carnet de contacts interactif**

**Objectifs pédagogiques** : POO, `ArrayList`, `Scanner`, `try-catch`, fichiers

## Description :

Créer une application **console** qui permet :

1. D'ajouter un contact (nom, prénom, numéro).
2. D'afficher tous les contacts.
3. De sauvegarder les contacts dans un fichier texte.
4. De charger les contacts au démarrage du programme.

## Concepts mobilisés :

- Classes `Contact` et `Carnet`
- Utilisation de `ArrayList<Contact>`
- Lecture/écriture avec `BufferedReader` / `BufferedWriter`
- Gestion d'entrées invalides avec `try-catch`

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

# 📓 Projet 2 : **Système de notes d'étudiants**

**Objectifs pédagogiques** : POO, `Map`, conditions, boucles, sérialisation (facultative)

## Description :

Créer une application **console** qui permet :

1. Permet d'ajouter un étudiant et sa note.
2. Affiche la liste des étudiants avec leurs moyennes.
3. Affiche les étudiants admis (note ≥ 10).
4. Sauvegarde les données dans un fichier `.ser` (bonus).

## Concepts mobilisés :

- Classe `Etudiant` (nom, note)
- Utilisation de `HashMap<String, Double>`
- Conditions ( `if` , `else` )
- (Optionnel) `ObjectOutputStream` / `ObjectInputStream`

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

# 🐶 Projet 3 : **Gestion d'un refuge pour animaux**

**Objectifs pédagogiques** : Héritage, polymorphisme, listes, méthodes, switch

## Description :

Créer une application **console** qui permet :

1. Gère différents types d'animaux ( `Chien` , `Chat` , etc.) héritant d'une classe `Animal` .
2. Permet d'ajouter un animal et de faire "parler" tous les animaux.
3. Utilise le polymorphisme pour appeler la méthode `communique()` .
4. Affiche les statistiques : combien de chats, de chiens, etc.

## Concepts mobilisés :

- Classe abstraite `Animal` , sous-classes `Chien` , `Chat`
- `ArrayList<Animal>` + boucle `for-each`
- Polymorphisme
- Utilisation de `instanceof` ou `getClass()` pour les stats

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

# Barème de notation

| Critère | Détail | Points |
|---|---|---|
| **Structure du code** | Classes claires, bonne organisation, noms explicites | **4** |
| **Fonctionnalités de base** | Fonctionnalités attendues bien implémentées | **6** |
| **Concepts Java maîtrisés** | POO, collections ( `List` , `Map` ), exceptions, fichiers | **4** |
| **Qualité du code** | Lisibilité, indentation, commentaires utiles | **2** |
| **Tests et robustesse** | Gestion des erreurs, entrées invalides, pas de crash | **2** |
| **Bonus / Initiative** | Améliorations personnelles, initiative, créativité | **2** |

🎯 **Total : /20**

*Le projet est évalué dans son ensemble : clarté, fonctionnalité, stabilité, et effort personnel.*

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

# 🤝 Merci de ton attention ! 🤝

Reste curieux 🧐 et amuse-toi ! 🎮 🎉

Ce cours t'a été présenté par

##  Moi-même, Nathaniel Vaur Henel 🤘

 mon site de partage autour du dév : dev-sensei.digicrafters.fr

 mon Framagit perso, GitLab éthique, libre & français : framagit.org/nathvh/me

 mon GitHub perso, mais inactif : github.com/Nathaniel-Vaur-Henel

 mon LinkedIn, pour suivre mon parcours pro : linkedin.com/in/nathaniel-vaur-henel

 mon Discord pro, pour échanger ou me poser des questions : `@nathanielvaurhenel`